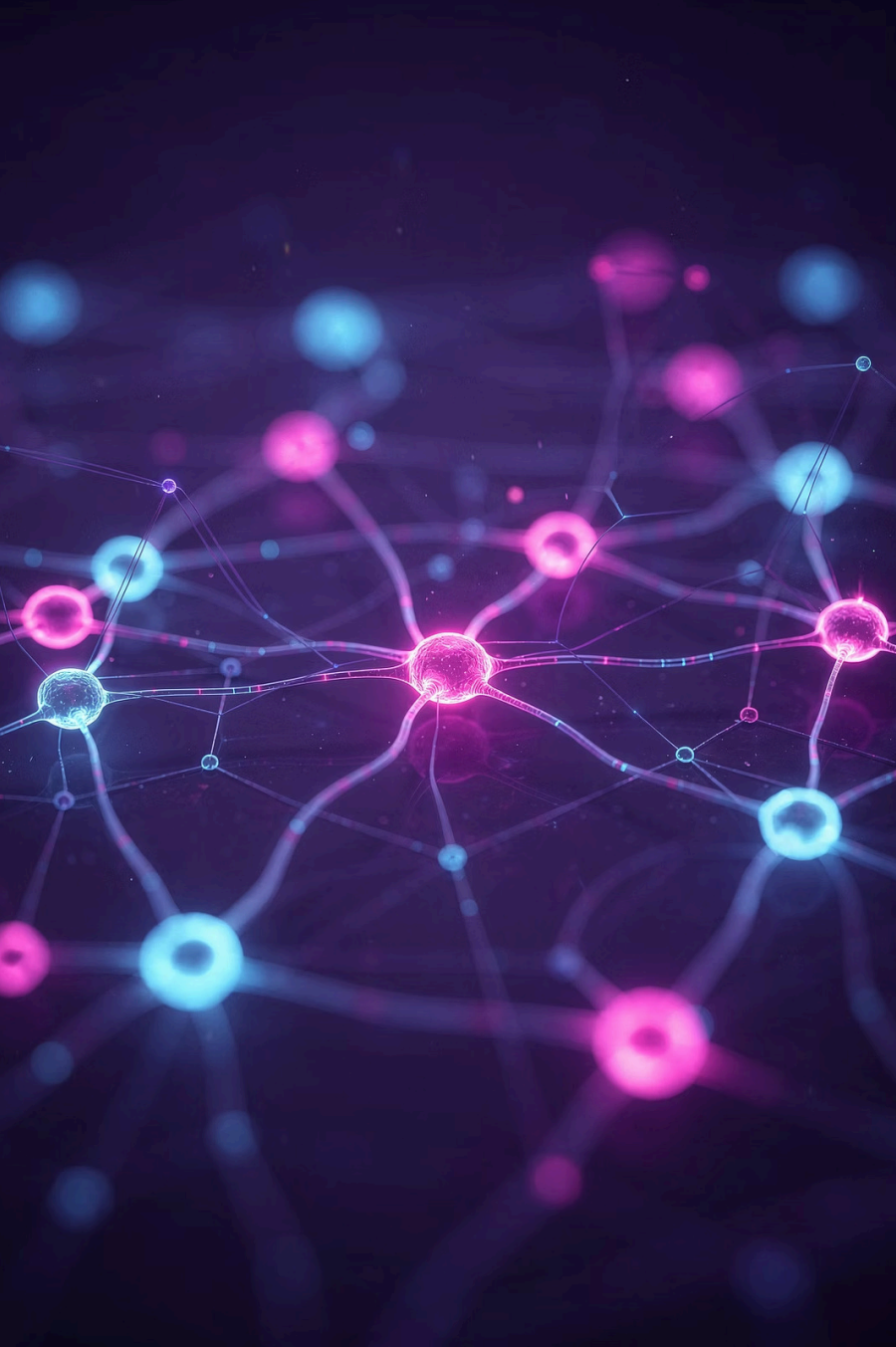




Reasoning Patterns and Observability

A practical guide to how AI agents think, act, and how you can see inside their decisions — before you start, take a moment to check where you stand.

AI AGENTS

OBSERVABILITY

EVALUATION

Before You Start

These aren't gates — they're anchors. Use them to locate yourself before diving in.

About Reasoning & Decision-Making

- Can you describe what a reasoning step looks like when a human recommends a product?
- Have you ever seen an AI response that jumped straight to an answer without explaining why?
- Do you understand roughly what a "loop" means in code — repeating a step until a condition is met?

About Observability

- Have you ever had to debug a system by reading logs after something went wrong?
- Do you know what a "trace" means in distributed systems — following a request across components?
- Have you looked at cost or latency dashboards for any API before?

About Testing & Evaluation

- Have you written any kind of test case — even informal ones — that verify a system's output?
- Do you know the difference between a unit test and an end-to-end behaviour test?

☐ Any gaps here are fine. The concepts ahead will fill them in.

Why Does an Agent Need to Reason?

A language model generates a response in one pass. An agent can **loop** — observe, think, act, observe again.

The Problem

A customer says *"I want a phone under \$300 with a good camera."* That's not a lookup — it's a multi-step decision.

Without Structured Reasoning

The agent guesses in one shot. It may miss constraints, ignore trade-offs, or return a confident but wrong answer.

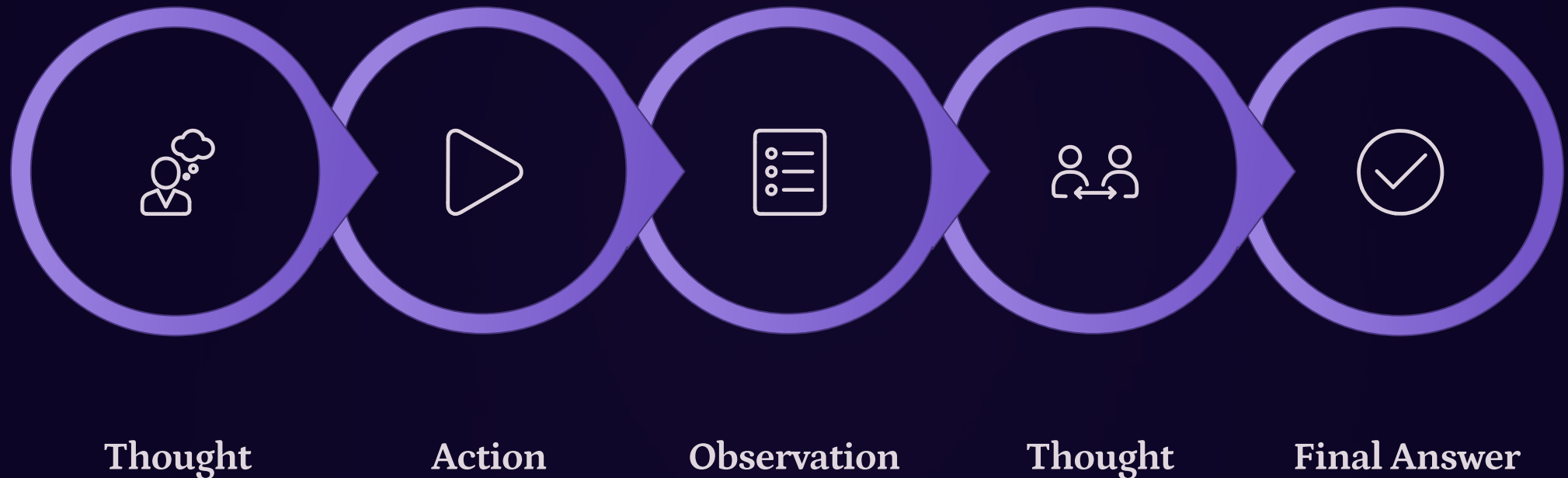
With Structured Reasoning

The agent works through the problem the way a person would: identify the constraint, retrieve candidates, compare them, then recommend.

The difference shows up most when the first retrieval result doesn't satisfy all constraints — a single-shot agent ignores that; a reasoning agent notices and adjusts.

What Is the ReAct Pattern?

ReAct stands for **Reason + Act** — the agent alternates between thinking about what to do and actually doing it. The model is not just generating text — it is making decisions, checking results, and deciding what to do next at each step.



This cycle ensures the agent's final answer is grounded in verified intermediate results — not a single confident guess.

ReAct vs. Single-Shot — When to Use Which

Common mistake: applying ReAct everywhere. Use it where the answer genuinely depends on intermediate steps.

Dimension	Single-Shot	ReAct Loop
Best for	Factual lookups, FAQ answers, simple structured queries	Multi-constraint decisions, dependent tool calls, iterative refinement
Latency	Low	Higher — each loop step adds time
Cost	Lower	Higher — more tokens per turn
Debuggability	Hard — one opaque answer	Easier — each step is inspectable

 For "What are your store hours?" — a single-shot response is the right call. Save the loop for where complexity demands it.

What Is the Reflection Pattern?

Reflection is when the agent **evaluates its own output** before returning it to the user — like reviewing a draft before sending.

Generate

After generating a recommendation, the agent produces its initial answer based on available context.

Evaluate

The agent asks itself: *"Does this actually satisfy the user's stated constraints?"* — checking price, specs, requirements.

Revise

"I recommended a phone at \$320 but the budget was \$300. I need to adjust." The agent corrects before the user ever sees the answer.

Reflection is most valuable when the stakes of a wrong first answer are high — wrong product, wrong price tier, mismatched requirement. It adds a self-correction layer **on top of** ReAct rather than replacing it.

Anti-Patterns in Reasoning Loops

Getting reasoning loops wrong makes an agent slower, more expensive, and harder to trust.

Infinite Loops

The termination condition is undefined or unreachable. The agent keeps calling tools with no exit. **Always set a max_iterations limit and a fallback response when it's hit.**

Redundant Tool Calls

The agent calls the same tool multiple times with identical inputs because it doesn't track what it has already done. Cache results within a run and check before re-calling.

Reasoning Without Acting

The agent produces long chains of thought but never commits to a tool call or final answer. This often signals an underspecified system prompt or missing termination signal.

Over-Reflection

Reflection is applied to every step, even trivial ones. The agent spends more tokens second-guessing simple lookups than it would have spent just answering. Reserve reflection for high-stakes outputs.

Context Overflow

The agent accumulates too much history in its context window across many steps. Older observations crowd out newer ones. Summarise or prune intermediate steps before they fill the window.

Most of these patterns are invisible until you have traces. Observability is what turns a mysterious failure into a diagnosable one.

Making Reasoning Visible

Users and developers both benefit from seeing how a decision was reached. This is not about dumping raw model output into the UI — it is about surfacing **structured, human-readable decision steps** that map to what the agent actually did.

For Users

A collapsible "**Agent Reasoning**" panel in the UI shows the thought steps without cluttering the main chat flow.

Visible reasoning builds trust: the agent is showing its work, not hiding it behind a confident-sounding sentence.

For Developers

Developers can trace each Thought → Action → Observation step in logs and traces.

- Pinpoint exactly where a wrong answer originated
- Audit decision chains after the fact
- Identify which steps are slow or expensive
- Reproduce edge cases reliably

What Is Observability for AI Agents?

Observability means you can understand what happened inside your system **after the fact** — without needing to change the code to find out.



Key Questions

Which agent ran? Which tools were called? Which model was used? How long did it take? What did it cost?



Beyond Traditional Logging

Traditional logging captures errors. Agent observability captures the **full decision trace** — inputs, routing choices, tool results, intermediate steps, and final outputs.



Why It Matters

Without observability, debugging a wrong answer in a multi-agent system means guessing which of many possible steps went wrong.

If a user got a wrong product recommendation today, how would you find out which step caused it?



What Does LangSmith Trace?

LangSmith captures the **full execution tree** of an agent run — not just the final input and output. This means you can reconstruct any agent run end-to-end after the fact, which is the foundation for both debugging and systematic improvement.



Nested Execution Tree

Each run shows the parent call (the orchestrator) and every child call — sub-agents, tool invocations, RAG retrievals — as a nested tree.



Per-Step Metrics

For each step: input payload, output, model name, latency in milliseconds, token count, and cost.



Tool Call Records

Tool calls are recorded with the exact arguments passed and the exact value returned — so you can see precisely what the agent handed to a tool and what came back.

Cost and Latency Awareness

LLM calls are not free and not instant. Observability helps you understand both dimensions **before they become problems**.

Cost Tracking

Which model is being called for which query type? Is the expensive model handling FAQ lookups that a lighter model could answer just as well?

Latency Tracking

Where is the agent actually slow — RAG retrieval, the tool call, or LLM generation? Pinpoint the bottleneck, not just the symptom.

Per-Session Cost

Over a full multi-turn conversation, what is the total spend? Aggregate visibility prevents billing surprises at scale.

The Routing Payoff

Good routing from the LiteLLM gateway reduces cost — but **only if you can measure** whether it is actually working in practice.

Observability closes the loop: you set up smart routing, then verify through traces and cost dashboards that the cheaper model is handling the right queries.

 Without measurement, optimization is just a guess.

What Is Agent Evaluation?

Evaluation answers a harder question than testing: **"Is the agent behaving correctly across a representative set of scenarios?"**

1

Testing

Passes or fails on one exact case. Binary, deterministic, narrow.

2

Evaluation

Measures quality across many varied inputs. Probabilistic, broad, representative.

3

PromptFoo

Runs a defined set of inputs against the agent and checks whether outputs meet specified criteria — text, routing, cost range.

- ❗ Evaluation is especially important for agents because their outputs are probabilistic. The same prompt does not always produce the same answer, so single-test verification is not enough.



Designing a Good Eval Suite

What makes an eval suite genuinely useful rather than just a set of green checkmarks?

1

Cover Multiple Flows

Support flows (order lookup, complaints) and sales flows (recommendations, comparisons) should both be represented.

2

Include Edge Cases

Out-of-stock products, invalid order IDs, ambiguous intents that could route either way.

3

Test Routing, Not Just Answers

Does "Where is my order?" always reach the right agent — every time?

4

Use Negative Cases

Does the agent correctly say "I don't know" rather than fabricating a confident-sounding answer?

A suite of 10–15 well-chosen cases gives more signal than 50 cases that all exercise the happy path.

Reading Evaluation Results

Passing evals is not the goal — **understanding what they reveal** is. A failing case tells you where the agent's behaviour doesn't match the expected pattern. That's valuable data, not a failure.

What Failures Tell You

- Consistent failures in one scenario area — say, budget-constrained recommendations — point to a prompt or routing issue worth isolating
- Latency outliers in LangSmith often correlate with eval cases that triggered more reasoning steps than expected

The Feedback Loop

01

Run Evals

02

Identify Failure Patterns

03

Refine Prompts or Routing

04

Re-Run Evals

The suite improves alongside the agent.

Patterns That Scale Better

Some observability and evaluation decisions age well. Others become technical debt quickly.

Structured Logs Over Print Statements

Structured logs are queryable and filterable; print output is noise at any scale beyond a single developer.

Eval Suites in Version Control

Prompts change frequently; your evals should track those changes so you know when a prompt update quietly broke a behaviour.

Trace IDs on Every Run

Makes it possible to correlate a specific user complaint with a specific agent trace, hours or days later.

Separate Eval Environments

Run evals against a stable snapshot of the agent, not live production, so results are reproducible and comparable.

What to Explore Next

These questions are designed to push your thinking further — they don't have single right answers.



Confident but Wrong

If an agent reasons through a problem but still gives a confident wrong answer, how would you catch that before a user sees it?



Complete Traces

When a system has multiple agents, RAG, and external tools all interacting, what does a complete trace need to capture?



Model Selection

How would you decide which model to use for which query type if you could only compare cost and latency?



Eval Diminishing Returns

At what point does adding more eval cases stop improving confidence and start becoming noise?



Guardrails vs. Observability

How do guardrails and observability complement each other — and why does one not replace the other?



Measuring Real Improvement

If evaluation results improve over time, how would you know whether the agent improved or the tests became too easy?